

BEE 4750 Homework 5: Mixed Integer and Stochastic Programming

Name: Matthew Burgos

ID: mb2557

Due Date

Thursday, 12/04/24, 9:00pm

Overview

Instructions

- In Problem 1, you will use mixed integer programming to solve a waste load allocation problem.
- In Problem 2, you will formulate a stochastic optimization problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Desktop/BEE 4750/hw5-mattyb`

```

using JuMP
using HiGHS
using DataFrames
using GraphRecipes
using Plots
using Measures
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (24 points)

Three cities are developing a coordinated municipal solid waste (MSW) disposal plan. Three disposal alternatives are being considered: a landfill (LF), a materials recycling facility (MRF), and a waste-to-energy facility (WTE). The capacities of these facilities and the fees for operation and disposal are provided below.

- **LF:** Capacity 200 Mg, fixed cost \$2000/day, tipping cost \$50/Mg;
- **MRF:** Capacity 350 Mg, fixed cost \$1500/day, tipping cost \$7/Mg, recycling cost \$40/Mg recycled;
- **WTE:** Capacity 210 Mg, fixed cost \$2500/day, tipping cost \$60/Mg;

Transportation costs are \$1.5/Mg-km, and the relative distances between the cities and facilities are provided in the table below.

City/Facility	Landfill (km)	MRF (km)	WTE (km)
1	5	30	15
2	15	25	10
3	13	45	20
LF	-	32	18
MRF	32	-	15
WTE	18	15	-

The fixed costs associated with the disposal options are incurred only if the particular disposal option is implemented. The three cities produce 100, 90, and 120 Mg/day of solid waste, respectively, with the composition provided in the table below.

Component	% of total mass	Combustion ash (%)	MRF Recycling rate (%)
Food Wastes	15	8	0

Component	% of total mass	Combustion ash (%)	MRF Recycling rate (%)
Paper & Cardboard	40	7	55
Plastics	5	5	15
Textiles	3	10	10
Rubber, Leather	2	15	0
Wood	5	2	30
Yard Wastes	18	2	40
Glass	4	100	60
Ferrous	2	100	75
Aluminum	2	100	80
Other Metal	1	100	50
Miscellaneous	3	70	0

The information in the above table will help you determine the overall recycling and ash fractions. Note that the recycling residuals, which may be sent to either landfill or the WTE, have different ash content than the ash content of the original MSW. You will need to determine these fractions to construct your mass balance constraints.

Reminder: Use `round(x; digits=n)` to report values to the appropriate precision!

Problem 1.1

Based on the information above, calculate the overall recycling and ash fractions for the waste produced by each city.

To do this, I store all data from the table into a dictionary, and loop through the dictionary to calculate the weighted averages.

```
waste_data = Dict(
    "Food Wastes" => Dict(
        "percent_mass" => 15.0,
        "combustion_ash" => 8,
        "recycling_rate" => 0
    ),
    "Paper & Cardboard" => Dict(
        "percent_mass" => 40.0,
        "combustion_ash" => 7,
        "recycling_rate" => 55
    ),
    "Plastics" => Dict(
        "percent_mass" => 5.0,
```

```

        "combustion_ash" => 5,
        "recycling_rate" => 15
    ),
    "Textiles" => Dict(
        "percent_mass" => 3.0,
        "combustion_ash" => 10,
        "recycling_rate" => 10
    ),
    "Rubber, Leather" => Dict(
        "percent_mass" => 2.0,
        "combustion_ash" => 15,
        "recycling_rate" => 0
    ),
    "Wood" => Dict(
        "percent_mass" => 5.0,
        "combustion_ash" => 2,
        "recycling_rate" => 30
    ),
    "Yard Wastes" => Dict(
        "percent_mass" => 18.0,
        "combustion_ash" => 2,
        "recycling_rate" => 40
    ),
    "Glass" => Dict(
        "percent_mass" => 4.0,
        "combustion_ash" => 100,
        "recycling_rate" => 60
    ),
    "Ferrous" => Dict(
        "percent_mass" => 2.0,
        "combustion_ash" => 100,
        "recycling_rate" => 75
    ),
    "Aluminum" => Dict(
        "percent_mass" => 2.0,
        "combustion_ash" => 100,
        "recycling_rate" => 80
    ),
    "Other Metal" => Dict(
        "percent_mass" => 1.0,
        "combustion_ash" => 100,
        "recycling_rate" => 50
    )

```

```

    ),
    "Miscellaneous" => Dict(
        "percent_mass" => 3.0,
        "combustion_ash" => 70,
        "recycling_rate" => 0
    )
)

overall_recycling_rate = 0
overall_ash_fraction = 0
for component in values(waste_data)
    overall_recycling_rate += component["percent_mass"]/100 * component["recycling_rate"]
    overall_ash_fraction += component["percent_mass"]/100 * component["combustion_ash"]
end

println("Overall recycling rate: ", overall_recycling_rate, "%")
println("Overall ash fraction: ", overall_ash_fraction, "%")

nonrecycled_waste_data = deepcopy(waste_data)
recycling_residuals_ash_fraction = 0
for component in values(nonrecycled_waste_data)
    component["percent_mass"] = component["percent_mass"] * (1-component["recycling_rate"])/100
    recycling_residuals_ash_fraction += component["percent_mass"]/100 * component["combustion_ash"]
end

println()
println("For later parts:")
println("Recycling residuals ash fraction: ", round(recycling_residuals_ash_fraction; digits=2), "%")

```

Overall recycling rate: 37.75%

Overall ash fraction: 16.41%

For later parts:

Recycling residuals ash fraction: 13.86%

Problem 1.2

What are the decision variables for your optimization problem? Provide notation and variable meaning.

Same as decision variables as the lecture slides:

W_{ij} = Waste transported from city i to disposal j (Mg/day)

R_{kj} = Residual waste transported from disposal k to disposal j (Mg/day)

Y_j = Operational status (on/off) of disposal j (binary)

Problem 1.3

Formulate the objective function. Make sure to include any needed derivations or justifications for your equation(s).

Same objective as the lecture slides:

Minimize total cost

Minimize transportation costs + disposal costs

Let

a_{ij} = Cost of transporting waste from source i to disposal j (\$/Mg-km)

l_{ij} = Distance between source i and disposal j (km)

c_j = Fixed costs of operating disposal j (\$/day)

b_j = Variable cost of disposing waste at disposal j (\$/Mg)

Then

$$\text{transportation costs} = \sum_{i \in I, j \in J} a_{ij} l_{ij} W_{ij}$$

$$\text{disposal costs} = \sum_{j \in J} [c_j Y_j + b_j \sum_i W_{ij}]$$

Objective

$$\min_{W_{ij}, Y_j} \sum_i \sum_j a_{ij} l_{ij} W_{ij} + \sum_j [c_j Y_j + b_j \sum_i W_{ij}]$$

Inputting Values

$$\text{WTE disposal costs} = 2500Y_{WTE} + 60(W_{1,WTE} + W_{2,WTE} + W_{3,WTE} + R_{MRF,WTE})$$

$$\text{MRF disposal costs} = 1500Y_{MRF} + 7(W_{1,MRF} + W_{2,MRF} + W_{3,MRF}) + 0.3775(40)(W_{1,MRF} + W_{2,MRF} + W_{3,MRF})$$

$$\text{LF disposal costs} = 2000Y_{LF} + 50(W_{1,LF} + W_{2,LF} + W_{3,LF} + R_{MRF,LF} + R_{WTE,LF})$$

$$\begin{aligned} \text{Transportation costs} = & 1.5(15W_{1,WTE} + 10W_{2,WTE} + 20W_{3,WTE} + 30W_{1,MRF} + 25W_{2,MRF} \\ & + 45W_{3,MRF} + 5W_{1,LF} + 15W_{2,LF} + 13W_{3,LF} + 15R_{MRF,WTE} + 32R_{MRF,LF} + 18R_{WTE,LF}) \end{aligned}$$

Final objective (calculating total cost and simplifying)

$$\begin{aligned} \min_{W_{ij}, R_{ij}, Y_j} & 2500Y_{WTE} + 1500Y_{MRF} + 2000Y_{LF} + 82.5W_{1,WTE} + 75W_{2,WTE} + 90W_{3,WTE} + 67.1W_{1,MRF} \\ & + 59.6W_{2,MRF} + 89.6W_{3,MRF} + 57.5W_{1,LF} + 72.5W_{2,LF} + 69.5W_{3,LF} + 82.5R_{MRF,WTE} + 98R_{MRF,LF} + 77R_{WTE,LF} \end{aligned}$$

Problem 1.4

Derive all relevant constraints. Make sure to include any needed justifications or derivations.

There are 5 categories of constraints

- 1) Indicator variables are 1 or 0 depending on if the facility is operating

$$Y_j = \begin{cases} 0 & \text{if } \sum_{i \in I} W_{ij} + \sum_{k \neq j} R_{kj} = 0 \\ 1 & \text{if } \sum_{i \in I} W_{ij} + \sum_{k \neq j} R_{kj} > 0 \end{cases}$$

2) All waste from each source needs to be disposed

$$\sum_i W_{ij} + \sum_k R_{kj} \leq K_j$$

3) Non-recycled and residual ash mass balance

$$R_{MRF,WTE} + R_{MRF,LF} = \text{recycling}\% * 5(W_{1,MRF} + W_{2,MRF} + W_{3,MRF})$$

$$R_{WTE,LF} = \text{ash}\% * (W_{1,WTE} + W_{2,WTE} + W_{3,WTE}) + \text{residual ash}\% * (R_{MRF,WTE})$$

4) Disposal facilities can't go above capacity

$$\sum_j W_{ij} = S_i$$

5) Non-negative constraint

$$W_{ij}, R_{ij} \geq 0$$

Inputting values

1.

$$Y_{WTE} = \begin{cases} 0 & \text{if } W_{1,WTE} + W_{2,WTE} + W_{3,WTE} + R_{MRF,WTE} = 0 \\ 1 & \text{else} \end{cases}$$

$$Y_{MRF} = \begin{cases} 0 & \text{if } W_{1,MRF} + W_{2,MRF} + W_{3,MRF} = 0 \\ 1 & \text{else} \end{cases}$$

$$Y_{LF} = 1$$

2.

$$W_{1,WTE} + W_{1,MRF} + W_{1,LF} = 100$$

$$W_{2,WTE} + W_{2,MRF} + W_{2,LF} = 90$$

$$W_{3,WTE} + W_{3,MRF} + W_{3,LF} = 120$$

3.

$$R_{MRF,WTE} + R_{MRF,LF} = 0.6225(W_{1,MRF} + W_{2,MRF} + W_{3,MRF})$$

$$R_{WTE,LF} = 0.1641(W_{1,WTE} + W_{2,WTE} + W_{3,WTE}) + 0.1386(R_{MRF,WTE})$$

4.

$$W_{1,WTE} + W_{2,WTE} + W_{3,WTE} + R_{MRF,WTE} \leq 210$$

$$W_{1,MRF} + W_{2,MRF} + W_{3,MRF} \leq 350$$

$$W_{1,LF} + W_{2,LF} + W_{3,LF} + R_{MRF,LF} + R_{WTE,LF} \leq 200$$

5.

$$W_{ij}, R_{ij} \geq 0$$

Problem 1.5

Find the optimal solution (using JuMP to solve the problem). Report the optimal objective value.

```
# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

# Sets
i = 1:3                # waste types: W1, W2, W3
fac = [:WTE, :MRF, :LF] # facilities

# -----
# Decision Variables
# -----
@variables(model, begin
    W[i, fac] >= 0        # waste flows
    R_MRF_WTE >= 0        # residuals from MRF to WTE
    R_MRF_LF  >= 0        # residuals from MRF to LF
    R_WTE_LF  >= 0        # residuals from WTE to LF
    Y_WTE, Bin
    Y_MRF, Bin
    Y_LF == 1             # specified in problem
end)

# -----
# Objective
# -----
@objective(model, Min,
    2500*Y_WTE
    + 1500*Y_MRF
```

```

+ 2000*Y_LF
+ 82.5*W[1,:WTE] + 75*W[2,:WTE] + 90*W[3,:WTE]
+ 67.1*W[1,:MRF] + 59.6*W[2,:MRF] + 89.6*W[3,:MRF]
+ 57.5*W[1,:LF] + 72.5*W[2,:LF] + 69.5*W[3,:LF]
+ 82.5*R_MRF_WTE
+ 98*R_MRF_LF
+ 77*R_WTE_LF
)

# -----
# Mass Balance
# -----
@constraint(model, W[1,:WTE] + W[1,:MRF] + W[1,:LF] == 100)
@constraint(model, W[2,:WTE] + W[2,:MRF] + W[2,:LF] == 90)
@constraint(model, W[3,:WTE] + W[3,:MRF] + W[3,:LF] == 120)

# -----
# Corrected Recycling Relations
# -----
@constraint(model,
    R_MRF_WTE + R_MRF_LF ==
        0.6225 * (W[1,:MRF] + W[2,:MRF] + W[3,:MRF])
)

@constraint(model,
    R_WTE_LF ==
        0.1641 * (W[1,:WTE] + W[2,:WTE] + W[3,:WTE]) +
        0.1386 * R_MRF_WTE
)

# -----
# Facility capacity limits
# -----
@constraint(model,
    W[1,:WTE] + W[2,:WTE] + W[3,:WTE] + R_MRF_WTE 210)

@constraint(model,
    W[1,:MRF] + W[2,:MRF] + W[3,:MRF] 350)

@constraint(model,
    W[1,:LF] + W[2,:LF] + W[3,:LF] + R_MRF_LF + R_WTE_LF 200)

```

```

# -----
# Binary Y Logic
# -----
M = 1e5

# WTE activation
@constraint(model,
    sum(W[i,:WTE] for i in 1:3) + R_MRF_WTE    M * Y_WTE)
@constraint(model,
    sum(W[i,:WTE] for i in 1:3) + R_MRF_WTE    1e-6 * Y_WTE)

# MRF activation
@constraint(model,
    sum(W[i,:MRF] for i in 1:3)    M * Y_MRF)
@constraint(model,
    sum(W[i,:MRF] for i in 1:3)    1e-6 * Y_MRF)

# -----
# Solve
# -----
optimize!(model)

# -----
# Output: ONLY objective + variable values
# -----
println("Objective Value = ", objective_value(model))
for v in all_variables(model)
    println(name(v), " = ", value(v))
end

```

```

Objective Value = 27855.48211508554
W[1,WTE] = 0.0
W[2,WTE] = 90.0
W[3,WTE] = 41.594688359851666
W[1,MRF] = 0.0
W[2,MRF] = -0.0
W[3,MRF] = 0.0
W[1,LF] = 100.0
W[2,LF] = -0.0
W[3,LF] = 78.40531164014834
R_MRF_WTE = 0.0
R_MRF_LF = 0.0

```

$R_{WTE_LF} = 21.59468835985166$
 $Y_{WTE} = 1.0$
 $Y_{MRF} = -0.0$
 $Y_{LF} = 1.0$

This was solved by inputting images of the linear program into ChatGPT, and adjusting the ChatGPT code accordingly.

Optimal total cost: \$27,855

Problem 1.6

Draw a diagram showing the flows of waste between the cities and the facilities. Which facilities (if any) will not be used? Does this solution make sense?

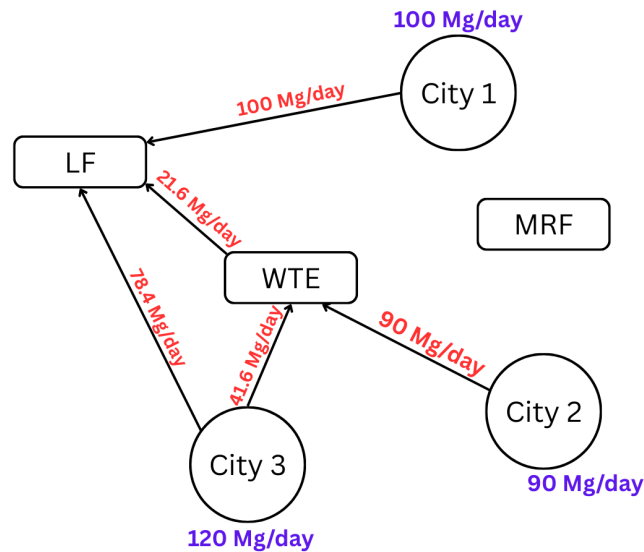


Figure 1: Flows of waste between cities and facilities

The materials recycling facility (MRF) is not used. This may be surprising because the MRF has the lowest tipping cost and fixed cost. However, it makes sense because only 37.75% of the waste is recyclable. This means that over 60% of the mass that reaches the MRF would have to go somewhere else to actually be disposed of. This would add extra transportation costs and tipping costs. In addition, the tipping costs of non-recyclables arriving at the MRF makes it so using the MRF doesn't make sense.

Problem 2 (6 points)

Consider a two-period economic dispatch problem, based on the multi-period example from Lecture 14 (on 10/29). The generator data, including ramping constraints for each generator, is provided in ‘data/generators.csv.’ In period 1, the demand is $d_1 = 1100\text{MW}$. In period 2, the demand is projected to be $d_2 = 1200\text{MW}$, but there is a 25% probability that it is 1500 . In the first period, the solar capacity factor is 0.9 and the wind capacity factor is 0.45, but in the second period, there is some uncertainty: the forecasted solar and wind capacity factors are 0.95 and 0.4, respectively, but there is a 30% probability that they are 0.75 and 0.5. Your goal is to identify how to dispatch your generators to minimize the cost of meeting demand.

Problem 2.1

Draw a scenario tree for this problem.

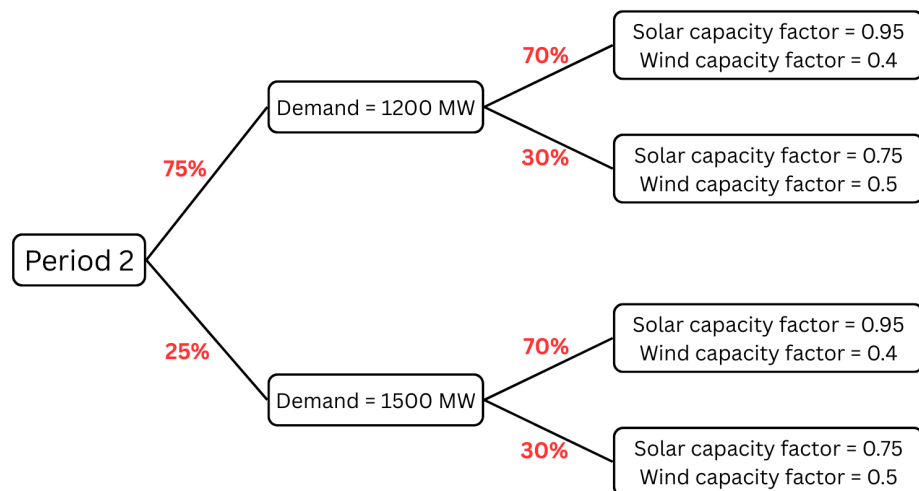


Figure 2: Scenario tree for 2nd period of economic dispatch problem

Problem 2.2

Formulate a stochastic linear program for this problem based on your scenario tree from Problem 2.1 and the data in `data/generators.csv`.

Let scenarios 1-4 represent the scenarios from top to bottom on the scenario tree respectively

decision variables:

$y_{g,t,s}$ = generation (MWh) by generator g at period t and scenario s

$$y_{bio,1}, y_{hydro,1}, y_{geo,1}, y_{ccgt,1}, y_{ct,1}, y_{wind,1}, y_{solar,1}$$

$$y_{bio,2,1}, y_{hydro,2,1}, y_{geo,2,1}, y_{ccgt,2,1}, y_{ct,2,1}, y_{wind,2,1}, y_{solar,2,1}$$

$$y_{bio,2,2}, y_{hydro,2,2}, y_{geo,2,2}, y_{ccgt,2,2}, y_{ct,2,2}, y_{wind,2,2}, y_{solar,2,2}$$

$$y_{bio,2,3}, y_{hydro,2,3}, y_{geo,2,3}, y_{ccgt,2,3}, y_{ct,2,3}, y_{wind,2,3}, y_{solar,2,3}$$

$$y_{bio,2,4}, y_{hydro,2,4}, y_{geo,2,4}, y_{ccgt,2,4}, y_{ct,2,4}, y_{wind,2,4}, y_{solar,2,4}$$

objective

minimize total expected variable cost across both periods

minimize (variable cost in period 1 + expected variable cost in period 2)

$$\min_{y_{g,t,s}} \sum_g VarCost_g * y_{g,1} + probability_s \sum_g VarCost_g * y_{g,2,s}$$

$$\min_{y_{g,t}} 5y_{bio,1} + 23y_{ccgt,1} + 38y_{ct,1}$$

$$+(0.75)(0.7)[5y_{bio,2,1} + 23y_{ccgt,2,1} + 38y_{ct,2,1}]$$

$$+(0.75)(0.3)[5y_{bio,2,2} + 23y_{ccgt,2,2} + 38y_{ct,2,2}]$$

$$+(0.25)(0.7)[5y_{bio,2,3} + 23y_{ccgt,2,3} + 38y_{ct,2,3}]$$

$$+(0.25)(0.3)[5y_{bio,2,4} + 23y_{ccgt,2,4} + 38y_{ct,2,4}]$$

constraint 1: demand must be met for all periods and scenarios

$$\sum_g y_{g,1} \geq 1100$$

$$\sum_g y_{g,2,1} \geq 1200$$

$$\sum_g y_{g,2,2} \geq 1200$$

$$\sum_g y_{g,2,3} \geq 1500$$

$$\sum_g y_{g,2,4} \geq 1500$$

constraint 2: generation cannot go above max generation limit * capacity factor

$$y_{bio,t,s} \leq 100$$

$$y_{hydro,t,s} \leq 500$$

$$y_{geo,t,s} \leq 400$$

$$y_{ccgt,t,s} \leq 500$$

$$y_{ct,t,s} \leq 250$$

$$y_{wind,1} \leq 300 * 0.45$$

$$y_{wind,2,1} \leq 300 * 0.4$$

$$y_{wind,2,2} \leq 300 * 0.5$$

$$y_{wind,2,3} \leq 300 * 0.4$$

$$y_{wind,2,4} \leq 300 * 0.5$$

$$y_{solar,1} \leq 500 * 0.9$$

$$y_{solar,2,1} \leq 500 * 0.95$$

$$y_{solar,2,2} \leq 500 * 0.75$$

$$y_{solar,2,3} \leq 500 * 0.95$$

$$y_{solar,2,4} \leq 500 * 0.75$$

constraint 3: generation cannot go below min generation limit

$$y_{bio,t,s} \geq 0$$

$$y_{hydro,t,s} \geq 0$$

$$y_{geo,t,s} \geq 0$$

$$y_{ccgt,t,s} \geq 220$$

$$y_{ct,t,s} \geq 100$$

$$y_{wind,t,s} \geq 0$$

$$y_{solar,t,s} \geq 0$$

constraint 4: next years generation cannot increase by more than ramping amount

$$\begin{aligned}y_{bio,2,s} - y_{bio,1} &\leq 100 \\y_{hydro,2,s} - y_{hydro,1} &\leq 500 \\y_{geo,2,s} - y_{geo,1} &\leq 400 \\y_{ccgt,2,s} - y_{ccgt,1} &\leq 100 \\y_{ct,2,s} - y_{ct,1} &\leq 200 \\y_{wind,2,s} - y_{wind,1} &\leq 300 \\y_{solar,2,s} - y_{solar,1} &\leq 500\end{aligned}$$

constraint 5: next years generation cannot decrease by more than ramping amount

$$\begin{aligned}y_{bio,2,s} - y_{bio,1} &\leq -100 \\y_{hydro,2,s} - y_{hydro,1} &\leq -500 \\y_{geo,2,s} - y_{geo,1} &\leq -400 \\y_{ccgt,2,s} - y_{ccgt,1} &\leq -100 \\y_{ct,2,s} - y_{ct,1} &\leq -200 \\y_{wind,2,s} - y_{wind,1} &\leq -300 \\y_{solar,2,s} - y_{solar,1} &\leq -500\end{aligned}$$

```
model = Model(HiGHS.Optimizer)
set_silent(model)

# =====
# SETS
# =====
generators = [:bio, :hydro, :geo, :ccgt, :ct, :wind, :solar]
periods = [1, 2]
scenarios = [1, 2, 3, 4] # used only for period 2

# =====
# PARAMETERS
# =====
```



```

# variable cost ($/MWh)
VarCost = Dict(
    :bio => 5,
    :hydro => 0,
    :geo => 0,
    :ccgt => 23,
    :ct => 38,
    :wind => 0,
    :solar => 0
)

# scenario probabilities (from image)
prob = Dict(1 => 0.75*0.7,
            2 => 0.75*0.3,
            3 => 0.25*0.7,
            4 => 0.25*0.3)

# demands
demand = Dict(
    (1,1)=>1100,
    (2,1)=>1200,
    (2,2)=>1200,
    (2,3)=>1500,
    (2,4)=>1500
)

# max generation limits
maxcap = Dict(
    :bio => 100,
    :hydro => 500,
    :geo => 400,
    :ccgt => 500,
    :ct => 250,
    :wind => 300,
    :solar => 500
)

# capacity factors in each scenario (wind & solar only)
capfactor = Dict(
    (:wind,1)=>0.45, (:wind,2)=>0.40, (:wind,3)=>0.50, (:wind,4)=>0.40,
    (:solar,1)=>0.90,(:solar,2)=>0.95,(:solar,3)=>0.95,(:solar,4)=>0.75
)

```

```

# ramp limits (increase or decrease) for each generator
ramp = Dict(
    :bio => 100,
    :hydro => 500,
    :geo => 400,
    :ccgt => 100,
    :ct => 200,
    :wind => 300,
    :solar => 500
)

# min generation
min_gen = Dict(
    :bio => 0,
    :hydro => 0,
    :geo => 0,
    :ccgt => 220,
    :ct => 100,
    :wind => 0,
    :solar => 0
)

# =====
# DECISION VARIABLES
# =====
@variable(model, y[g in generators, t in periods, s in (t==1 ? [1] : scenarios)] >= 0)

# =====
# OBJECTIVE
# =====
@objective(model, Min,
    # period 1
    sum(VarCost[g] * y[g,1,1] for g in generators)
    +
    # expected period 2 cost
    sum(prob[s] * sum(VarCost[g] * y[g,2,s] for g in generators) for s in scenarios)
)

# =====
# CONSTRAINT 1: Demand in all periods/scenarios
# =====
@constraint(model, sum(y[g,1,1] for g in generators) >= 1100)

```

```

@constraint(model, sum(y[g,2,1] for g in generators) >= 1200)
@constraint(model, sum(y[g,2,2] for g in generators) >= 1200)
@constraint(model, sum(y[g,2,3] for g in generators) >= 1500)
@constraint(model, sum(y[g,2,4] for g in generators) >= 1500)

# =====
# CONSTRAINT 2: Max generation
# =====
for g in generators
    # period 1 has no scenario CF
    @constraint(model, y[g,1,1] <= maxcap[g])
end

# wind & solar use scenario-dependent caps in period 2
for s in scenarios
    @constraint(model, y[:wind,2,s] <= maxcap[:wind] * capfactor[:wind,s])
    @constraint(model, y[:solar,2,s] <= maxcap[:solar] * capfactor[:solar,s])
end

# other generators use their fixed max limit
for g in (:bio, :hydro, :geo, :ccgt, :ct)
    for s in scenarios
        @constraint(model, y[g,2,s] <= maxcap[g])
    end
end

# =====
# CONSTRAINT 3: Min generation
# =====
for g in generators
    @constraint(model, y[g,1,1] >= min_gen[g])
    for s in scenarios
        @constraint(model, y[g,2,s] >= min_gen[g])
    end
end

# =====
# CONSTRAINT 4: Ramping up
# =====
for g in generators
    for s in scenarios
        @constraint(model, y[g,2,s] - y[g,1,1] <= ramp[g])
    end
end

```

```

        end
    end

# =====
# CONSTRAINT 5: Ramping down
# =====
for g in generators
    for s in scenarios
        @constraint(model, y[g,1,1] - y[g,2,s] <= ramp[g])
    end
end

# =====
# SOLVE
# =====
optimize!(model)

# =====
# OUTPUT (ONLY objective + decision variables)
# =====
println("Objective Value = ", objective_value(model))

for g in generators
    print("$g: ")
    for t in periods
        if t == 1
            # only scenario 1 exists in period 1
            v = value(y[g,1,1])
            print("y[$g,1,1]=$round(v, digits=4)) ")
        else
            # all 4 scenarios in period 2
            for s in scenarios
                v = value(y[g,2,s])
                print("y[$g,2,$s]=$round(v, digits=4)) ")
            end
        end
    end
    println() # new line for next generator
end

```

Objective Value = 17720.0

```

bio: y[bio,1,1]=0.0 y[bio,2,1]=0.0 y[bio,2,2]=0.0 y[bio,2,3]=0.0 y[bio,2,4]=0.0
hydro: y[hydro,1,1]=500.0 y[hydro,2,1]=500.0 y[hydro,2,2]=500.0 y[hydro,2,3]=500.0 y[hydro,2,4]=500.0
geo: y[geo,1,1]=400.0 y[geo,2,1]=400.0 y[geo,2,2]=400.0 y[geo,2,3]=400.0 y[geo,2,4]=400.0
ccgt: y[ccgt,1,1]=220.0 y[ccgt,2,1]=220.0 y[ccgt,2,2]=220.0 y[ccgt,2,3]=220.0 y[ccgt,2,4]=220.0
ct: y[ct,1,1]=100.0 y[ct,2,1]=100.0 y[ct,2,2]=100.0 y[ct,2,3]=100.0 y[ct,2,4]=100.0
wind: y[wind,1,1]=300.0 y[wind,2,1]=135.0 y[wind,2,2]=120.0 y[wind,2,3]=150.0 y[wind,2,4]=150.0
solar: y[solar,1,1]=500.0 y[solar,2,1]=450.0 y[solar,2,2]=475.0 y[solar,2,3]=475.0 y[solar,2,4]=475.0

```

This was solved by inputting images of the linear program into ChatGPT, and adjusting the ChatGPT code accordingly.

References

List any external references consulted, including classmates.

<https://latexeditor.lagrida.com/>

<https://chatgpt.com/>

<https://envsys.viveks.me/fall2025/slides/lecture12-1-waste-management.html>

<https://envsys.viveks.me/fall2025/slides/lecture13-1-stochastic-optimization.html>

<https://envsys.viveks.me/fall2025/slides/lecture10-2-economic-dispatch.html>